Hierarchical Facility Location Problem – Report

Daniele Bugatti & Giovanni Gaio

June 8, 2026

1 Introduction

The presence and distribution of medical facilities within a geographical region address a critical objective: maximizing accessibility to ensure that every resident can reach a healthcare provider in the shortest possible time. Historically, however, the development of these services has been carried out independently by local authorities, each focusing solely on their respective territories without a large-scale, coordinated strategy. Consequently, this uncoordinated growth has led, for example, to an oversupply of hospitals relative to actual demand. In contrast, we propose a top-down strategy that enables holistic planning. By considering both population distribution and the road network, this approach aims to minimize the average travel distance for residents to their nearest facility. Furthermore, we account for the fact that hospital facilities vary in their capacity and the breadth of services they offer. Specifically, we distinguish two hierarchical levels: local clinics, which can be distributed more densely across the territory but cannot address all medical needs, and hospital hubs, which are more resource-intensive and thus fewer in number, but can fulfill any medical requirement, including those covered by local clinics.

In our project, we model this challenge as a graph optimization problem. Given a fixed number k_1 and k_2 of Level 1 and Level 2 facilities, respectively, the objective is to select two subsets of vertices, S_1 and S_2, subject to the following constraints:

$$\begin{cases} |S_1| = k_1 \\ |S_2| = k_2 \\ S_2 \subseteq S_1 \end{cases}$$

The third condition defines the hierarchical nature of the facilities, establishing this problem as an extension of the classic Facility Location Problem: Level 2 facilities, being higher in the hierarchy, inherently encompass the capabilities of Level 1 facilities. The selection of S_1 and S_2 aims to minimize the following objective function:

$$\sum_{i \in V} h_i \min_{j \in S_1} d_{ij} + \sum_{i \in V} h_i \min_{j \in S_2} d_{ij}$$

Since the resulting optimization problem exhibits a combinatorial search space that grows exponentially with the number of candidate locations, exact approaches become computationally intractable for large-scale instances. Therefore, we investigate two heuristic methods—a simulated annealing algorithm and a genetic algorithm—comparing their performance with the exact solution computed via an ILP solver on both synthetic and real datasets.

2 Methods

In this project, we developed two approximate algorithms to tackle the complexity of the HFLP search space: one based on the simulated annealing technique and another implementing a genetic algorithm. The core principle behind both methods is to progressively improve candidate solutions through non-deterministic updates that, on average, lower the objective function value. While this approach enables an efficient, guided exploration of a fraction of the search space, it does not guarantee the optimality of the final solution.

2.1 Simulated Annealing

We implemented a Simulated Annealing algorithm where, for each move, a single facility chosen at random from both S_1 and S_2 and it placed in another empty node. Of course, if the facility extracted has Level 2, its new location will be chosen among $S_1 \setminus S_2$.

Then, the acceptance criterion follows the standard

$$P(S \to S') = \begin{cases} 1 & \text{if } \Delta Z < 0 \\ e^{-\frac{\Delta Z}{T}} & \text{if } \Delta Z \geq 0 \end{cases}$$

where Δ is the difference of the objective function in the new and the old configuration.

The temperature follows the bounded geometric progression $T_{k+1} = \min(\alpha T_k, T_{\text{MIN}})$. In our algorithm we fixed $\alpha = 0.995$ and $T_{\text{MIN}} = 10^{-3}$.

2.2 Genetic Algorithm

We further implemented a Genetic Algorithm (GA) to metaheuristically explore the HFLP solution space. Each individual in the population is represented as a tuple of sets (S_1, S_2) corresponding to a feasible configuration of open facilities. The fitness of an individual is directly defined by its total demand-weighted distance, which the algorithm aims to minimize.

To transition from one generation to the next, the population is evolved through three sequential operators:

- **Selection**: A k-way tournament selection is applied. For each parent slot, k individuals are sampled uniformly at random from the population, and the one exhibiting the lowest objective function value wins the tournament.
- **Crossover**: With a predefined crossover probability, two parents combine their genetic material. The offspring's Level-1 set (S_1) is formed by randomly sampling p_1 nodes from the union of the parents' Level-1 sets. To ensure the nesting constraint, the Level-2 set (S_2) is constructed by sampling p_2 nodes from the intersection of the newly formed S_1 and the union of the parents' Level-2 sets. If this intersection contains fewer than p_2 candidates, the remaining slots are filled by sampling directly from S_1.
- **Mutation**: With a mutation rate μ, the offspring undergoes independent random swap mutations. A Level-1 mutation replaces an open site in S_1 with a closed one (propagating the change to S_2 if the removed node was nested). Similarly, a Level-2 mutation swaps a site in S_2 with an available node in $S_1 \setminus S_2$.

Generational replacement is performed alongside an elitism strategy, ensuring that the best-performing individual from the previous generation always survives unchanged into the next.

3 Result

We compared the quality of the results of our methods with a standard ILP solver.

3.1 Synthetic data

As a preliminary step we tested on two families of synthetic graphs: random graphs (Erdos-Reyni) and grid graphs. We tested first small graphs, with 50 nodes, then larger graphs with up to 400 nodes.

For the cases where it was possible we compared the solutions from SA and GA with the optimal given by the ILP. For smaller graphs, up to 100 nodes, we observed that SA always found the optimal solution with runtime similar to ILP. GA also achieved an optimal cost in most cases, though with longer runtime.

On larger graphs, where we couldn't evaluate the optimal ILP solution in reasonable times, we observed similar optimized cost for SA and GA. It is notable that SA found consistently better solutions than GA on a roughly equivalent runtime.

3.2 Simulation on Veneto

In this case we chose $p_1 = 21$, equal to the number of hospitals in the region. $p_2 = 5$ is chosen for no specific reason.

Surprisingly, on this graph we were also able to get a solution using ILP.